



Báo cáo cuối kỳ XLA final

Xử lý ảnh (Trường Đại học Sư phạm Kỹ Thuật Thành phố Hồ Chí Minh)



messages.pdf_cover_qr_code_label

BỘ GIÁO DỤC & ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN TỰ ĐỘNG ĐIỀU KHIỂN

-----Δ-----



BÁO CÁO CUỐI KỲ
XỬ LÝ ẢNH
ĐỀ TÀI: NHẬN DIỆN CỬ CHỈ TAY

GVHD: TS. DƯƠNG MINH THIÊN

SVTH:

MSSV:

HÀ VĂN TÂN

22151144

HOÀNG DUY PHƯƠNG

22151136

Tp. Hồ Chí Minh, ngày 15 tháng 06 năm 2025

BÁO CÁO CUỐI KỲ
HỌC KÌ II NĂM 2024 – 2025

- 1. **Mã lớp môn học:** IMPR432446
- 2. **Giảng viên hướng dẫn:** TS. Dương Minh Thiện
- 3. **Tên đề tài:** Hệ thống đếm số lần cho động tác ngồi xổm (Squat)
- 4. **Danh sách sinh viên thực hiện:**

STT	Họ và tên	MSSV	Hoàn thành	Ghi chú
1	Hà Văn Tân	22151144	100%	
2	Hoàng Duy Phương	22151136	100%	

Nhận xét của giảng viên:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Chữ kí của giảng viên

TS. Dương Minh Thiện

LỜI CẢM ƠN

Nhóm chúng em xin được viết lời cảm ơn chân thành nhất đến thầy Dương Minh Thiện, thầy đã dành thời gian, kiến thức và tình cảm quý báu để hướng dẫn và đánh giá báo cáo cuối kỳ của nhóm chúng em. Đây là một cơ hội quý giá để nhóm có thể học hỏi, trau dồi kiến thức và phát triển kỹ năng trong lĩnh vực mà chúng em đam mê.

Báo cáo cuối kỳ này không thể hoàn thành một cách thành công và trọn vẹn nếu như không có sự hướng dẫn tận tình và chân thành của thầy. Từ những khóa học, buổi hướng dẫn và đưa ra lời khuyên quý giá, thầy đã truyền đạt những kiến thức sâu sắc và phương pháp làm việc chuyên nghiệp, giúp chúng em vượt qua khó khăn và tiến bộ trong quá trình thực hiện báo cáo.

Bên cạnh những kiến thức chuyên môn, em còn học được cách làm việc khoa học, logic và tinh thần tự nghiên cứu, tự học hỏi để hoàn thiện các vấn đề phát sinh trong quá trình thực hiện báo cáo. Những điều này thực sự là trải nghiệm quý giá, giúp nhóm em tự tin hơn khi làm việc trong các dự án sau này.

Cuối cùng, chúng em xin chúc thầy Dương Minh Thiện sức khỏe dồi dào, thành công trong sự nghiệp giảng dạy và nghiên cứu. Mong rằng những kiến thức và kinh nghiệm tuyệt vời của thầy sẽ được chia sẻ và truyền đạt tiếp cho các thế hệ sinh viên tiếp theo.

Xin chân thành cảm ơn!

Trân trọng

Mục lục

Mục lục iv

<i>Danh sách hình ảnh</i>	<i>vi</i>
<i>Danh sách bảng</i>	<i>vii</i>
<i>Chương 1. TỔNG QUAN</i>	<i>4</i>
1.1 Đặt vấn đề	4
1.2 Mục tiêu đề tài	5
1.3 Phương pháp nghiên cứu	6
1.4 Giới hạn đề tài	6
1.5 Nội dung nghiên cứu	6
<i>Chương 2. CƠ SỞ LÝ THUYẾT</i>	<i>8</i>
2.1 MEDIAPIPE	8
2.1.1 Giới thiệu về Mediapipe	8
2.1.2 Ứng dụng Mediapipe trong Hand gesture detection	9
2.2 Làm mịn GAUSSIAN	10
2.3 Tính toán góc giữa ba điểm (góc tại đầu gối)	11
2.4 Nội suy tuyến tính	13
2.4.1 Khái niệm	13
2.4.2 Công thức	13
2.4.3 Những lưu ý cần nắm để sử dụng công thức nội suy chính xác	14
<i>Chương 3. XÂY DỰNG CHƯƠNG TRÌNH</i>	<i>15</i>
3.1 Giới thiệu về Python	15
3.2 Lưu đồ giải thuật	16
3.3 Mô tả các bước xử lý	16
3.3.1 Khởi tạo camera và đọc ảnh từ webcam	16
3.3.2 Tải ảnh mẫu tương ứng số ngón tay	16
3.3.3 Khởi tạo bộ phát hiện bàn tay (Mediapipe wrapper)	17
3.3.4 Tìm bàn tay và các điểm mốc (landmarks)	17
3.3.5 Tạo danh sách trạng thái từng ngón tay	21
3.3.6 Đếm số ngón tay đang giơ lên	21
3.3.7 Hiển thị ảnh minh họa tương ứng	23
3.3.8 Hiển thị số ngón tay và FPS	23
3.3.9 Vòng lặp xử lý	23
3.4 Chương trình thực hiện	23
<i>Chương 4. KẾT QUẢ VÀ KẾT LUẬN</i>	<i>28</i>

4.1	Kết quả đạt được	28
4.2	Kết luận.....	30
4.3	Hướng phát triển	31
<i>TÀI LIỆU THAM KHẢO.....</i>		<i>35</i>

Danh sách hình ảnh

<i>Hình 2.1: Các giải pháp được cung cấp và tính khả dụng trên các nền tảng</i>	<i>9</i>
<i>Hình 2.2: Mô hình 20 điểm quan trọng của tay</i>	<i>9</i>
<i>Hình 2.3: Nội suy tuyến tính khá phổ biến</i>	<i>13</i>
<i>Hình 3.1: Lưu đồ giải thuật</i>	<i>16</i>
<i>Hình 3.2: Ảnh mẫu các ngón tay</i>	<i>17</i>
<i>Hình 4.1: Hiển thị kết quả</i>	<i>28</i>
<i>Hình 4.2: Hiển thị kết quả</i>	<i>29</i>
<i>Hình 4.3: Hiển thị kết quả</i>	<i>30</i>

Danh sách bảng

*Bảng 1: Pose Estimation (Phát hiện tư thế người)..... **Error! Bookmark not defined.***

Chương 1. TỔNG QUAN

1.1 Đặt vấn đề

Xử lý ảnh là một lĩnh vực giao thoa giữa khoa học và công nghệ, với tốc độ phát triển nhanh chóng trong những thập kỷ gần đây. Mặc dù còn tương đối mới so với nhiều ngành khoa học truyền thống, nhưng xử lý ảnh đã trở thành một phần không thể thiếu trong các ứng dụng hiện đại như y tế, giao thông, nông nghiệp, an ninh giám sát, và đặc biệt là tương tác giữa người và máy tính.

Ban đầu, xử lý ảnh chủ yếu tập trung vào việc nâng cao chất lượng hình ảnh và phân tích nội dung ảnh. Một trong những ứng dụng đầu tiên có thể kể đến là việc cải thiện chất lượng ảnh báo truyền qua cáp từ Luân Đôn đến New York vào những năm 1920. Sau Thế chiến thứ hai, cùng với sự phát triển của máy tính, lĩnh vực xử lý ảnh số đã có những bước tiến quan trọng. Đặc biệt, vào năm 1964, công nghệ này đã được áp dụng để xử lý hình ảnh thu được từ tàu vũ trụ Ranger 7 của Mỹ, mở ra kỷ nguyên mới cho việc khai thác thông tin từ ảnh số.

Từ đó đến nay, xử lý ảnh không ngừng được cải tiến nhờ vào sự phát triển của các công cụ tính toán mạnh mẽ, thuật toán hiện đại, cũng như sự hỗ trợ của trí tuệ nhân tạo, đặc biệt là mạng nơ-ron nhân tạo. Các kỹ thuật này đã góp phần làm tăng hiệu quả trong việc phân tích, nhận dạng, và tương tác thông minh giữa con người với thiết bị.

Vì vậy, việc nghiên cứu và phát triển hệ thống NHẬN DIỆN CỬ CHỈ TAY không chỉ mang lại giá trị ứng dụng cao, mà còn góp phần vào xu hướng phát triển các hệ thống tương tác người - máy hiện đại, thông minh và thuận tiện hơn. Đây chính là động lực thúc đẩy việc triển khai đề tài này.

1.2 Mục tiêu đề tài

Mục tiêu chính của đề tài là nghiên cứu và xây dựng một hệ thống nhận diện cử chỉ tay có khả năng phát hiện và phân loại chính xác các cử chỉ tay trong thời gian thực, phục vụ cho việc điều khiển hoặc tương tác với máy tính, thiết bị điện tử hoặc hệ thống thông minh mà không cần tiếp xúc vật lý.

Cụ thể, đề tài hướng đến các mục tiêu sau:

1. Tìm hiểu và phân tích các phương pháp nhận diện cử chỉ tay hiện có: từ kỹ thuật truyền thống sử dụng xử lý ảnh (Image Processing) đến các phương pháp hiện đại dựa trên học máy (Machine Learning) và học sâu (Deep Learning).
2. Xây dựng hệ thống thu thập và xử lý ảnh đầu vào: sử dụng camera để lấy hình ảnh bàn tay trong thời gian thực và xử lý ảnh để phát hiện vùng chứa cử chỉ tay.
3. Thiết kế và huấn luyện mô hình phân loại cử chỉ tay: ứng dụng các kỹ thuật như CNN (Convolutional Neural Network) để nhận dạng các cử chỉ tay định nghĩa sẵn (ví dụ: OK, Stop, Like, Dislike, số đếm...).
4. Triển khai hệ thống nhận diện thời gian thực: đảm bảo độ chính xác cao, tốc độ xử lý nhanh, giao diện đơn giản và dễ sử dụng.
5. Đề xuất các ứng dụng tiềm năng: trình diễn điều khiển một chức năng cụ thể (như chuyển slide, điều khiển nhạc, hoặc điều khiển thiết bị IoT) bằng cử chỉ tay.

Thông qua việc đạt được các mục tiêu trên, đề tài góp phần xây dựng một hệ thống tương tác thông minh, mở rộng khả năng giao tiếp tự nhiên giữa con người và máy móc, đồng thời tạo nền tảng cho các nghiên cứu và ứng dụng nâng cao trong tương lai.

1.3 Phương pháp nghiên cứu

Để thực hiện đề tài "Nhận diện cử chỉ tay", nhóm nghiên cứu kết hợp giữa phương pháp nghiên cứu lý thuyết và phương pháp thực nghiệm. Trước hết, nhóm tiến hành thu thập và phân tích các tài liệu khoa học liên quan đến xử lý ảnh, thị giác máy tính và trí tuệ nhân tạo, đặc biệt là các kỹ thuật phát hiện và phân loại cử chỉ tay như mạng nơ-ron tích chập (CNN), MediaPipe, và các mô hình học sâu phổ biến. Trên cơ sở đó, hệ thống được xây dựng thông qua quá trình thực nghiệm gồm các bước: thu thập dữ liệu cử chỉ tay từ camera hoặc tập dữ liệu có sẵn; tiền xử lý ảnh như chuyển đổi không gian màu, cắt vùng tay, chuẩn hóa kích thước và tăng cường dữ liệu để cải thiện độ chính xác của mô hình. Sau đó, mô hình học sâu được huấn luyện để phân loại các cử chỉ tay định nghĩa trước. Cuối cùng, hệ thống được tích hợp và triển khai nhận diện cử chỉ tay trong thời gian thực, sử dụng thư viện OpenCV kết hợp với mô hình đã huấn luyện. Việc đánh giá được thực hiện dựa trên độ chính xác, tốc độ xử lý và tính ổn định trong môi trường thực tế, từ đó đề xuất cải tiến và hướng phát triển tiếp theo cho hệ thống.

1.4 Giới hạn đề tài

Đề tài tập trung vào việc nhận diện một tập hợp các cử chỉ tay tĩnh (static hand gestures) được định nghĩa trước, chẳng hạn như: giờ tay, nắm tay, chỉ một ngón, biểu tượng "OK", "Like", "Stop"... Hệ thống sử dụng camera 2D thông thường để thu nhận hình ảnh đầu vào và thực hiện xử lý ảnh kết hợp với mô hình học sâu (CNN hoặc MediaPipe) để phân loại cử chỉ. Nhằm đảm bảo tính khả thi và thời gian thực hiện, đề tài chưa mở rộng sang nhận diện cử chỉ động (dynamic gestures) hay ngôn ngữ ký hiệu liên tục. Ngoài ra, việc nhận diện được tiến hành trong điều kiện ánh sáng tương đối ổn định và nền không quá phức tạp. Hệ thống cũng mới dừng lại ở việc hiển thị kết quả nhận diện và chưa tích hợp sâu vào các ứng dụng thực tế như điều khiển thiết bị thông minh hay robot.

1.5 Nội dung nghiên cứu

Chương 1: Tổng quan

Chương 2: Cơ sở lý thuyết

Chương 3: Xây dựng chương trình

Chương 4: Kết quả và kết luận

Chương 2. CƠ SỞ LÝ THUYẾT

2.1 MEDIAPIPE

2.1.1 Giới thiệu về Mediapipe

MediaPipe là một framework xử lý thị giác máy tính đa nền tảng, được phát triển bởi Google, tích hợp nhiều giải pháp machine learning tối ưu hóa cho các ứng dụng thời gian thực. Đây là một công cụ nhẹ, dễ tùy biến và mã nguồn mở, cho phép người dùng triển khai các mô hình học máy hiệu quả trên nhiều nền tảng khác nhau như di động, máy tính, web, điện toán đám mây và các thiết bị IoT.

- Một số ưu điểm nổi bật của MediaPipe bao gồm:
 - + Khả năng suy luận (inference) nhanh và hiệu quả: MediaPipe được thiết kế để hoạt động ổn định trên hầu hết các cấu hình phần cứng phổ thông, đảm bảo hiệu suất xử lý thời gian thực ngay cả trên thiết bị di động.
 - + Dễ dàng cài đặt và triển khai: Với sự hỗ trợ đa nền tảng, MediaPipe cho phép triển khai linh hoạt trên các hệ điều hành như Android, iOS, Windows, Linux, cũng như các trình duyệt Web thông qua WebAssembly.
 - + Mã nguồn mở và miễn phí: Toàn bộ mã nguồn của MediaPipe được công khai, cho phép người dùng hoàn toàn tự do sử dụng, tùy chỉnh hoặc tích hợp vào các hệ thống hiện có để giải quyết các bài toán cụ thể một cách linh hoạt và hiệu quả.

	Android	iOS	C++	Python	JS	Coral
Face Detection	✓	✓	✓	✓	✓	✓
Face Mesh	✓	✓	✓	✓	✓	
Iris	✓	✓	✓			
Hands	✓	✓	✓	✓	✓	
Pose	✓	✓	✓	✓	✓	
Holistic	✓	✓	✓	✓	✓	
Selfie Segmentation	✓	✓	✓	✓	✓	
Hair Segmentation	✓		✓			
Object Detection	✓	✓	✓			✓
Box Tracking	✓	✓	✓			
Instant Motion Tracking	✓					
Objectron	✓		✓	✓	✓	
KNIFT	✓					
AutoFlip			✓			
MediaSequence			✓			
YouTube 8M			✓			

Hình 2.1: Các giải pháp được cung cấp và tính khả dụng trên các nền tảng

2.1.2 Ứng dụng Mediapipe trong Hand gesture detection

Với ứng dụng Hand gesture detection, ta có thể xây dựng một mô hình cơ thể người với các điểm như sau:



Hình 2.2: Mô hình 20 điểm quan trọng của tay

Khi bật camera, Pose sẽ xác định các điểm chính này trên tay bạn theo thời gian thực và tạo ra một hình ảnh biểu diễn chuyển động của con người giống như hình que. Nó nhận diện các điểm khớp trên tay và theo dõi chuyển động của chúng. Việc sử dụng Mediapipe trong Hand Gesture Detection (Phát hiện cử chỉ tay) có thể giúp

xây dựng các ứng dụng như điều khiển bằng cử chỉ, nhận diện ngôn ngữ ký hiệu, và các hệ thống tương tác không chạm.

Mỗi điểm gồm:

- x, y : tọa độ pixel chuẩn hóa theo chiều ngang và dọc ảnh.
- z : chiều sâu tương đối.
- $visibility$: độ tin cậy (giá trị từ 0 đến 1).

- Sử dụng MediaPipe Pose giúp hệ thống đạt được độ chính xác cao, tốc độ xử lý thời gian thực, và dễ triển khai trong các ứng dụng thể dục thông minh.

2.2 Làm mịn GAUSSIAN

Toán tử làm mịn Gaussian là một phương pháp xử lý ảnh được sử dụng để làm mờ hình ảnh, giúp loại bỏ chi tiết nhỏ và nhiễu. Gaussian hoạt động dựa trên phép tích chập hai chiều giữa hình ảnh gốc và một kernel Gaussian. Kernel này có dạng hình chuông và tuân theo phân bố Gaussian. So với bộ lọc trung bình, toán tử Gaussian sử dụng một kernel có đặc điểm khác biệt, giúp duy trì cấu trúc tổng thể của hình ảnh trong khi vẫn làm mờ các chi tiết nhỏ.

Công thức hàm Gaussian 2D:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \quad (2.1)$$

Với (x, y) là tọa độ pixel trong ma trận

σ là độ lệch chuẩn, xác định độ làm mờ

Ma trận kernel của bộ lọc Gaussian 2D

$$\begin{bmatrix} (-1, -1) & (-1, 0) & (-1, 1) \\ (0, -1) & (0, 0) & (0, 1) \\ (1, -1) & (1, 0) & (1, 1) \end{bmatrix} \quad (2.2)$$

Trong đó: Hàng tương ứng với giá trị y

Cột tương ứng với giá trị x

Gốc tọa độ (0, 0) được đặt ở giữa ma trận

Hàm này được ánh xạ thành một ma trận kernel để thực hiện tích chập lên ảnh đầu vào, mục đích của bước này là làm giảm nhiễu và các chi tiết nhỏ trong ảnh.

2.3 Tính toán góc giữa ba điểm (góc tại đầu gối)

Cho ba điểm trên mặt phẳng 2D:

- Điểm A: Đại diện cho hông (x_1, y_1) .
- Điểm B: Đại diện cho đầu gối (x_2, y_2) , là đỉnh của góc.
- Điểm C: Đại diện cho mắt cá chân (x_3, y_3) .

Chúng ta cần tính góc θ tại đầu gối (điểm B) được tạo bởi các vector BA (từ đầu gối đến hông) và BC (từ đầu gối đến mắt cá chân). Góc được tính bằng hàm atan2 , hàm này tính góc của một vector so với trục x dương. Hiệu số tuyệt đối giữa các góc của hai vector sẽ cho chúng ta góc tại đầu gối. Sau đó, góc này được chuẩn hóa để nằm trong khoảng $[0, 180]$ độ, vì đây thường là khoảng quan tâm cho các góc khớp trong sinh cơ học.

Công thức được cung cấp là:

$$\theta = |\text{atan2}(c - b) - \text{atan2}(a - b)| \quad (2.3)$$

Trong đó:

- $a - b$ là vector từ điểm B (đầu gối) đến điểm A (hông).
- $c - b$ là vector từ điểm B (đầu gối) đến điểm C (mắt cá chân).
- $\text{atan2}(y, x)$ tính góc của vector (x, y) so với trục x dương, trả về giá trị trong khoảng $[-\pi, \pi]$ radian.

Kết quả được chuyển đổi sang độ và chuẩn hóa về $[0, 180]$ để biểu thị góc bên trong tại đầu gối, giúp xác định các tư thế như ngồi (góc nhỏ, $\sim 90^\circ$) hoặc đứng (góc lớn, $\sim 180^\circ$).

Các bước cần tiến hành

- Bước 1: Đầu vào:

- + Mỗi điểm là một cặp tọa độ: $A(x_1, y_1)$, $B(x_2, y_2)$, $C(x_3, y_3)$.
- + Thông thường, các điểm này được lấy từ dữ liệu phân tích tư thế 2D (ví dụ: từ các mô hình thị giác máy tính).
- Bước 2: Tính vector:
 - + Vector $BA = (x_1 - x_2, y_1 - y_2)$.
 - + Vector $BC = (x_3 - x_2, y_3 - y_2)$.
- Bước 3: Tính góc bằng atan2:
 - + Sử dụng atan2 để tìm góc của mỗi vector so với trục x dương:
 - + Góc của BA: $\text{atan2}(y_1 - y_2, x_1 - x_2)$.
 - + Góc của BC: $\text{atan2}(y_3 - y_2, x_3 - x_2)$.
 - + atan2 trả về góc trong radian trong khoảng $[-\pi, \pi]$.
- Bước 4: Tính góc giữa hai vector:
 - + Góc θ giữa hai vector là hiệu số tuyệt đối giữa hai góc:

$$\theta = |\text{atan2}(y_3 - y_2, x_3 - x_2) - \text{atan2}(y_1 - y_2, x_1 - x_2)| \quad (2.4)$$
 - + Kết quả là góc trong radian.
- Bước 5: Chuyển đổi sang độ:
 - + Chuyển góc từ radian sang độ bằng cách:

$$\text{độ} = \text{radian} \cdot \frac{180}{\pi} \quad (2.5)$$
- Bước 6: Chuẩn hóa góc:
 - + Góc giữa hai vector nên nằm trong khoảng $[0, 180]$ độ cho góc bên trong tại đầu gối.
 - + Nếu góc tính được lớn hơn 180° , lấy 360° trừ đi góc đó để được góc nhỏ hơn:

$$\text{nếu } \theta > 180^\circ, \text{ thì } \theta = 360^\circ - \theta \quad (2.6)$$
 - + Hoặc, lấy $\min(\theta, 360 - \theta)$ để đảm bảo góc là góc nhỏ hơn.
- Bước 7: Diễn giải góc:
 - + Góc nhỏ (ví dụ: $\sim 90^\circ$ hoặc nhỏ hơn) thường cho thấy tư thế ngồi.
 - + Góc lớn (ví dụ: $\sim 180^\circ$) cho thấy tư thế đứng.
 - + Góc trung gian (ví dụ: $100^\circ - 160^\circ$) có thể là tư thế trung gian như crouching (gập người).

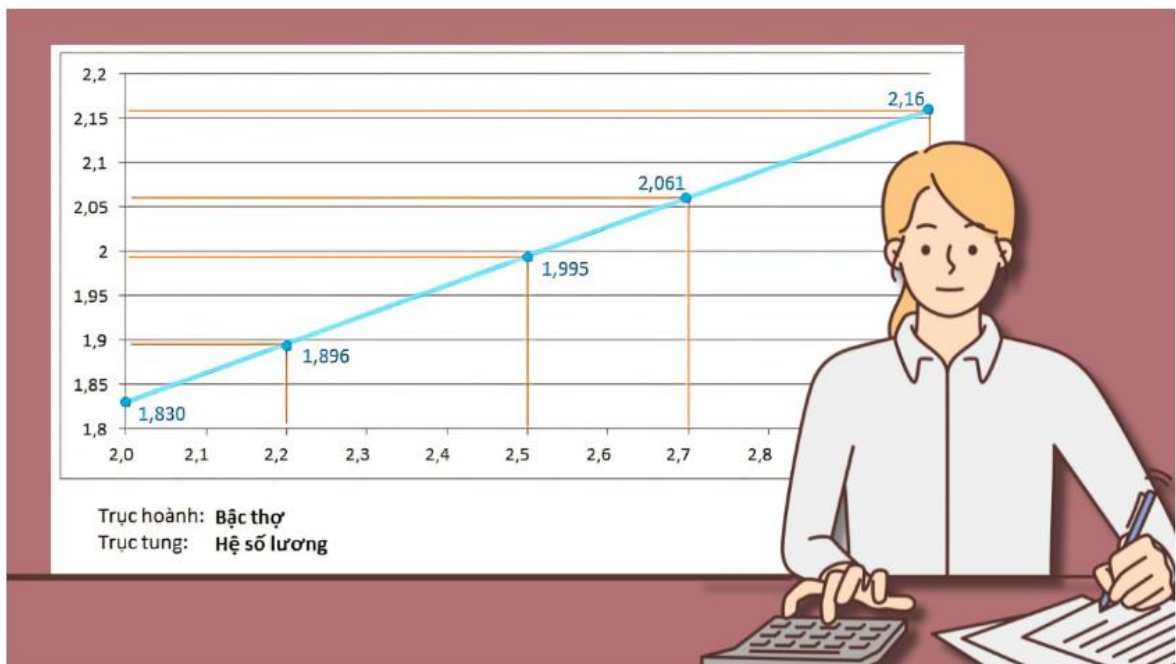
2.4 Nội suy tuyến tính

2.4.1 Khái niệm

Nội suy tuyến tính là một phương pháp hiệu quả để ước lượng giá trị của hàm số dựa trên tập dữ liệu đã có. Thay vì thực nghiệm hoặc đo đạc trực tiếp, các công thức nội suy giúp ta tính toán các giá trị một cách dễ dàng. Nhờ đó, nội suy trở thành công cụ quan trọng, hỗ trợ phân tích dữ liệu rời rạc và tối ưu hóa quá trình tính toán trong nhiều lĩnh vực.

2.4.2 Công thức

Nội suy tuyến tính là phương pháp khá phổ biến, được dùng để ước lượng giá trị hàm số. Nó được thực hiện theo nguyên tắc giả định sự thay đổi giữa hai điểm là tuyến tính.



Hình 2.3: Nội suy tuyến tính khá phổ biến

Dưới đây là công thức nội suy tuyến tính giữa (x_1, y_1) , (x_2, y_2) :

$$y = y_1 + \frac{(x - x_1)(y_2 - y_1)}{x_2 - x_1} \quad (2.7)$$

Trong đó:

- y : Giá trị nội suy.

- x : Điểm thực hiện nội suy.
- x_1, y_1 : Tọa độ thứ nhất.
- x_2, y_2 : Tọa độ thứ hai.

2.4.3 Những lưu ý cần nắm để sử dụng công thức nội suy chính xác

- Trước khi áp dụng công thức nội suy, hãy kiểm tra tính đồng nhất của dữ liệu để tránh các giá trị ngoại lai làm sai lệch kết quả.
- Lựa chọn phương pháp nội suy phù hợp với tính chất thông tin, chọn nội suy tuyến tính cho dữ liệu đơn giản, còn Spline hoặc đa thức cho các trường hợp phức tạp hơn.
- Hạn chế nội suy ngoài phạm vi dữ liệu có sẵn để tránh sai số.
- Đảm bảo lượng dữ liệu đủ lớn, vì nếu có quá ít thông tin thì có thể làm giảm độ chính xác của kết quả.

Chương 3. XÂY DỰNG CHƯƠNG TRÌNH

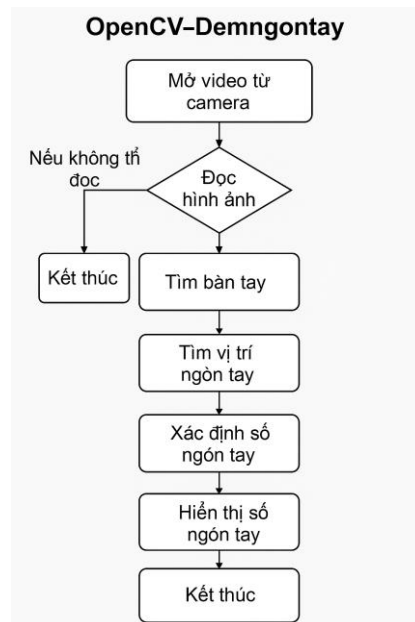
3.1 Giới thiệu về Python

Chương trình xây dựng bằng Python, một ngôn ngữ lập trình bậc cao, thông dịch và đa dụng. Python nổi bật với cú pháp rõ ràng, dễ học và khả năng hỗ trợ nhiều mô hình lập trình khác nhau, thiết kế với triết lý ưu tiên tính dễ đọc, giúp lập trình viên có thể viết mã nguồn một cách trực quan và dễ hiểu, đồng thời cung cấp sự linh hoạt trong việc phát triển phần mềm. Ngôn ngữ Python hỗ trợ lập trình hướng đối tượng, thủ tục và hàm. Cho phép người dùng lựa chọn phương pháp tiếp cận phù hợp với từng bài toán cụ thể.

Python được phát triển bởi Guido Van Rossum vào cuối thập niên 1980 và chính thức ra mắt vào tháng 2 năm 1991. Python nổi bật với thiết kế theo hướng mở rộng, thay vì tích hợp mọi tính năng vào phần lõi và cho phép mở rộng chức năng thông qua hệ thống mô-đun. Điều này giúp Python trở nên linh hoạt, dễ tùy chỉnh và thích hợp với nhiều mục đích phát triển khác nhau. Nhờ tính mô-đun nhỏ gọn nên Python còn được sử dụng để xây dựng giao diện lập trình cho các ứng dụng và phát triển những hệ thống phần mềm.

Hiện nay, Python là một trong những ngôn ngữ lập trình phổ biến nhất và rất thông dụng trong lĩnh vực khoa học và công nghệ. Với hệ sinh thái thư viện phong phú, Python đóng vai trò quan trọng trong nhiều lĩnh vực nghiên cứu và ứng dụng thực tiễn, bao gồm trí tuệ nhân tạo (AI), khoa học dữ liệu, thị giác máy tính (Computer Vision) và xử lý ảnh số (Digital Image Processing). Các thư viện như TensorFlow, PyTorch, OpenCV và NumPy,... đã giúp Python trở thành công cụ không thể thiếu trong việc phát triển các thuật toán học máy, phân tích dữ liệu và xử lý hình ảnh. Vì dễ sử dụng, khả năng mở rộng và cộng đồng phát triển mạnh mẽ, Python tiếp tục khẳng định vị thế của mình trong nghiên cứu khoa học và công nghệ hiện đại.

3.2 Lưu đồ giải thuật



Hình 3.1: Lưu đồ giải thuật

3.3 Mô tả các bước xử lý

Mọi quá trình xử lý hình ảnh đều được thực hiện trên từng khung hình của nguồn video hoặc camera do người dùng lựa chọn, kết hợp OpenCV để đọc và xử lý, NumPy để tính toán dữ liệu để hiển thị lên giao diện.

3.3.1 Khởi tạo camera và đọc ảnh từ webcam

```
“cap = cv2.VideoCapture(0)
```

```
ret, frame = cap.read()”
```

-Mở webcam và lấy từng khung hình từ video.

3.3.2 Tải ảnh mẫu tương ứng số ngón tay

```
“FolderPath = "Fingers"
```

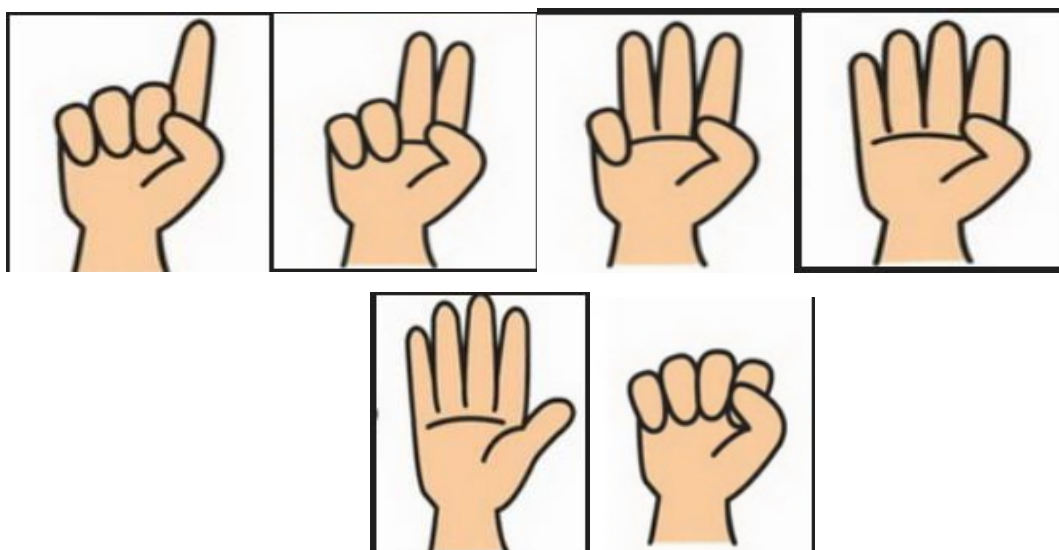
```
lst = os.listdir(FolderPath)
```

```
for i in lst:
```

```
    image = cv2.imread(f'{FolderPath}/{i}')
```

```
    lst_2.append(image)”
```

-Đọc các ảnh số ngón tay từ thư mục Fingers để hiển thị tương ứng với kết quả nhận diện.



Hình 3.2: Ảnh mẫu các ngón tay

3.3.3 Khởi tạo bộ phát hiện bàn tay (Mediapipe wrapper)

“detector = htm.handDetector(detectionCon=0.55)”

-Sử dụng module hand để khởi tạo đối tượng phát hiện bàn tay với độ tin cậy (confidence) là 0.55.

3.3.4 Tìm bàn tay và các điểm mốc (landmarks)

“frame = detector.findHands(frame)”

lmList = detector.findPosition(frame, handNo=0)”

-Chuyển ảnh về RGB, truyền vào Mediapipe để:

+Tìm các bàn tay trong ảnh.

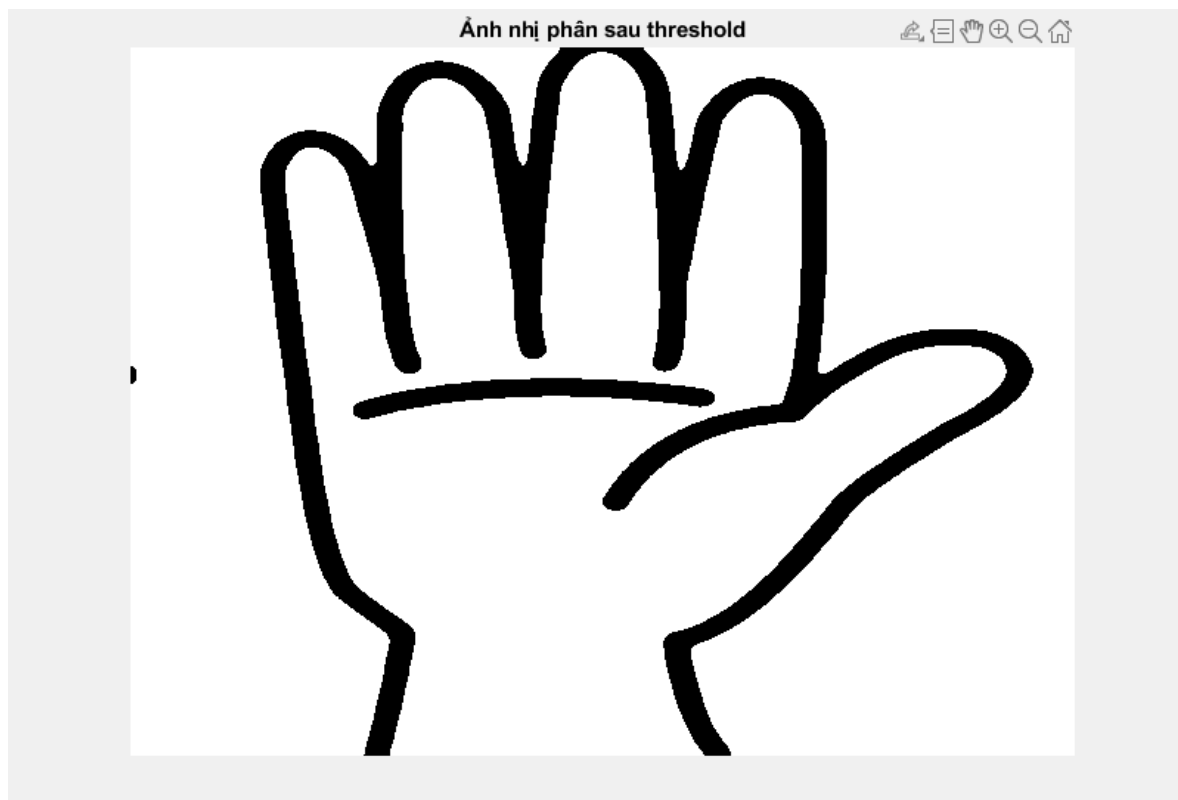
+Trích xuất vị trí các 20 điểm đặc trưng (landmarks) của bàn tay.



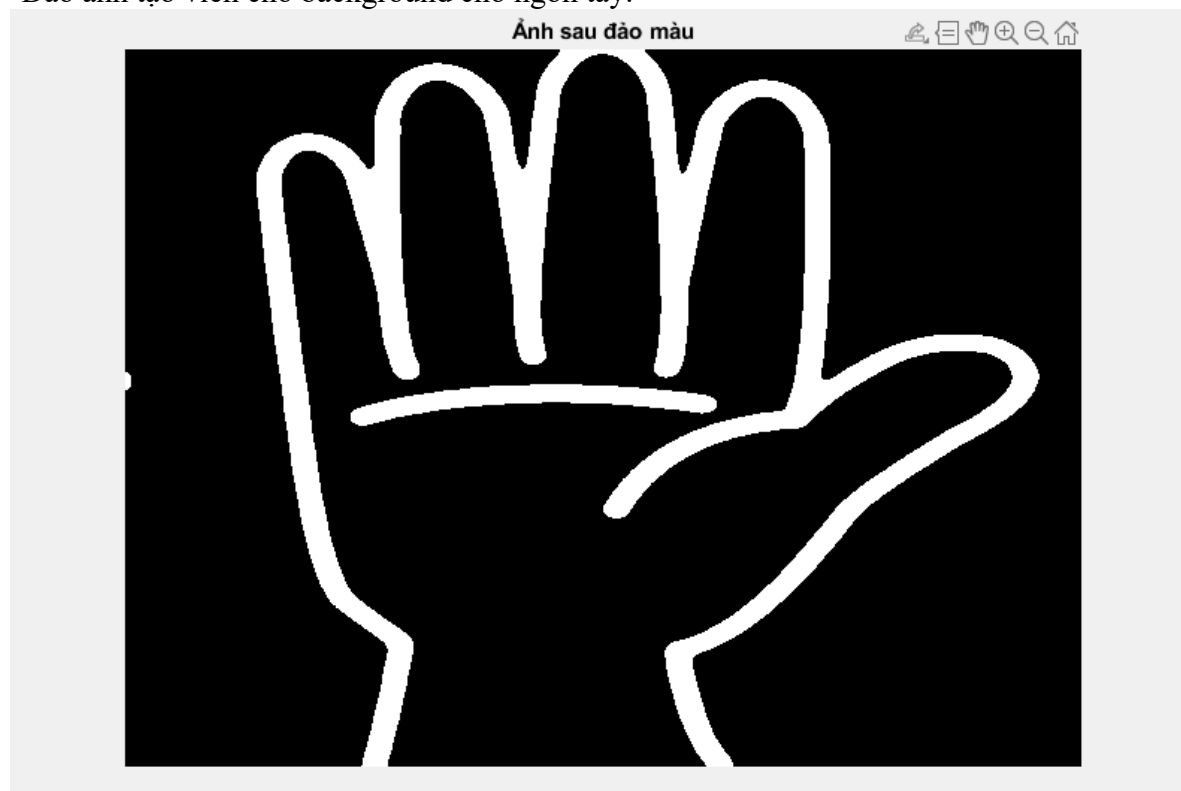
-Làm mịn ảnh bằng Gaussian.



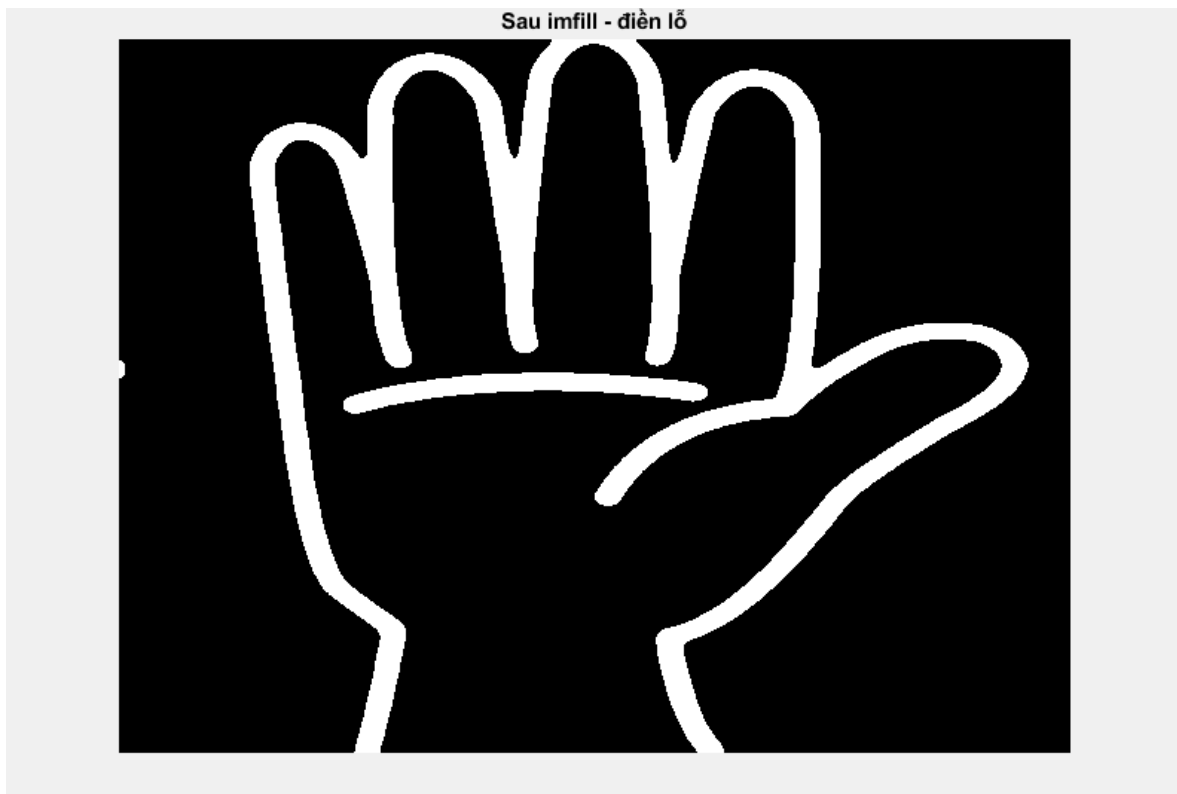
-Chuyển ảnh sang nhị phân.



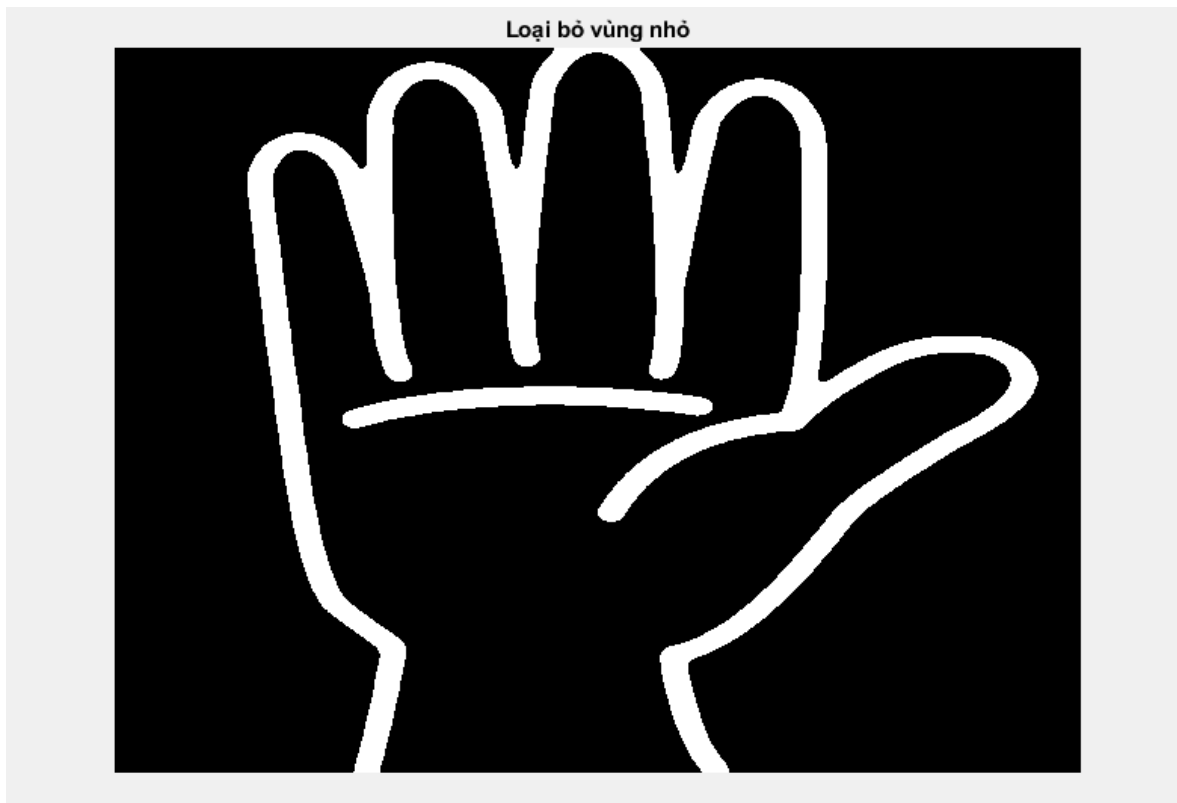
-Đảo ảnh tạo viền cho background cho ngón tay.



-Điền lỗ để làm mất các đường răng cưa và phân rõ ranh giới của background và foreground hơn.



-Loại bỏ nhiều những vùng trắng bên ngoài biến mất và làm tăng ranh giới giữa 2 vùng trong ảnh.



3.3.5 Tạo danh sách trạng thái từng ngón tay

```

“fingers = []
# Ngón cái
if lmList[fingerid[0]][1] < lmList[fingerid[0] - 1][1]:
    fingers.append(1)
else:
    fingers.append(0)
# 4 ngón còn lại
for id in range(1, 5):
    if lmList[fingerid[id]][2] < lmList[fingerid[id] - 2][2]:
        fingers.append(1)
    else:
        fingers.append(0)”

```

-Dựa trên vị trí tương đối của các điểm landmark:

+So sánh chiều ngang (trục x) cho ngón cái.

+So sánh chiều dọc (trục y) cho các ngón còn lại.

3.3.6 Đếm số ngón tay đang giơ lên

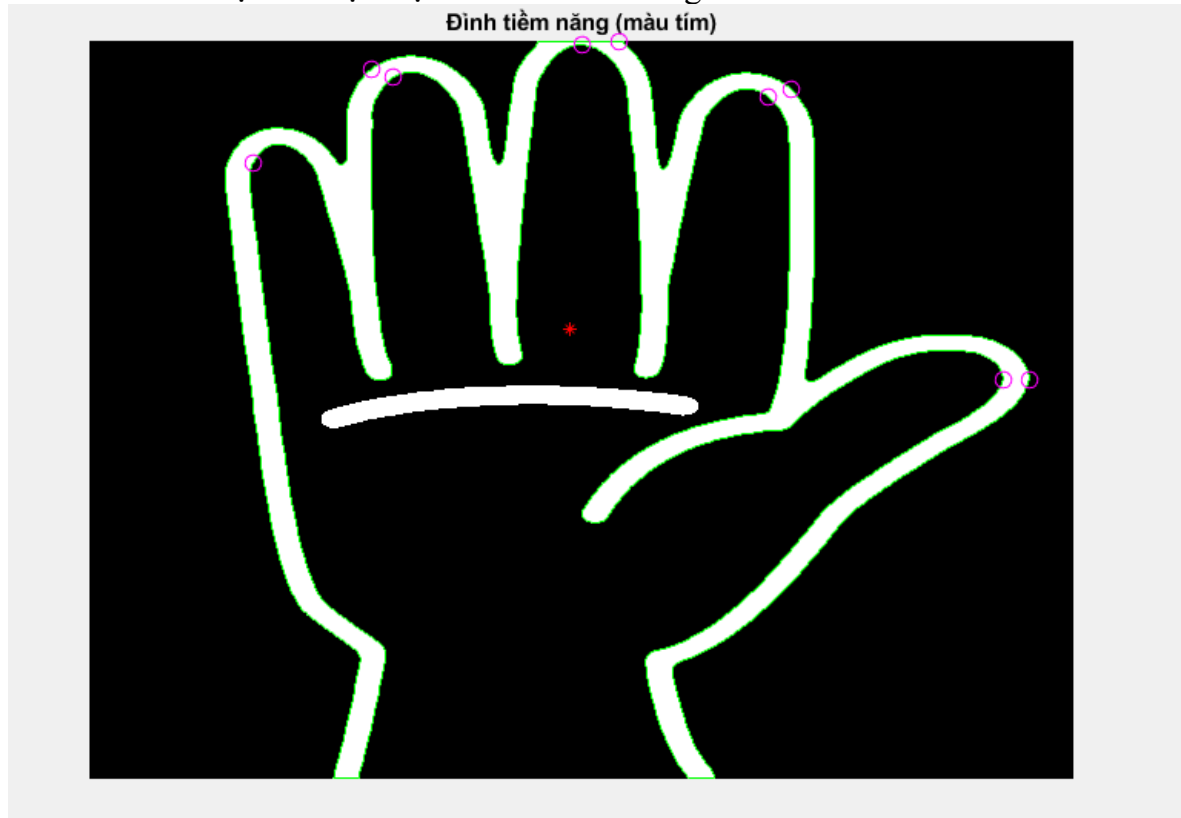
```

“songontay = fingers.count(1)”

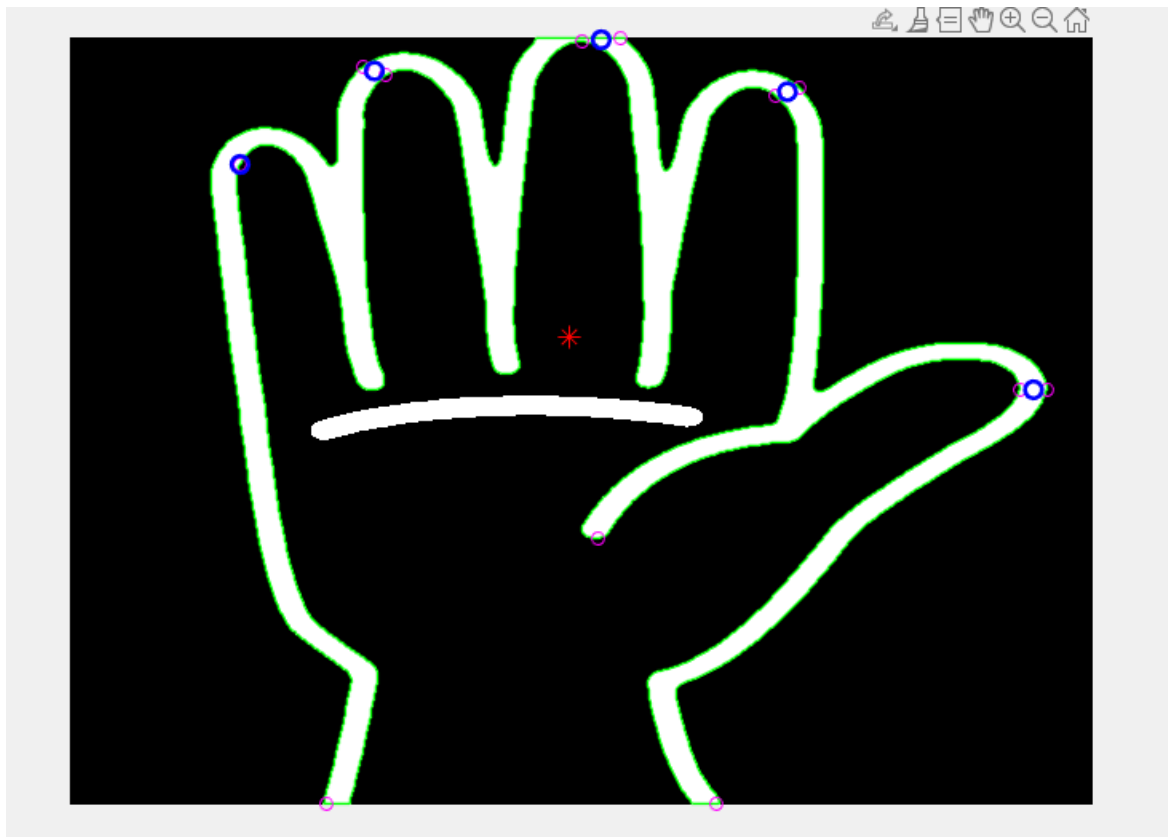
```

-Đếm số phần tử có giá trị 1 trong mảng fingers — tức là số ngón tay được giơ lên.

-Tính toán ma trận và hiện thị các điểm tiềm năng



-Thu gom trung bình các điểm tiềm năng và xác định tọa độ các ngón tay đang được mở ra.



-Xử lý toàn bộ và hiển thị các điểm được xác định và nhận dạng, chỉ ra tọa độ.



3.3.8 Hiển thị số ngón tay và FPS

3.3.9 Vòng lặp xử lý

3.4 Chương trình thực hiện

BỘ MÔN TỰ ĐỘNG ĐIỀU KHIỂN

```
import hand as htm

# Mở video từ camera
cap = cv2.VideoCapture(0)

FolderPath = "Fingers"
lst = os.listdir(FolderPath)

lst_2 = [] # Danh sách chứa các ảnh ngón tay
for i in lst:
    image = cv2.imread(f'{FolderPath}/{i}')
    if image is not None:
        lst_2.append(image)

pTime = 0.55

detector = htm.handDetector(detectionCon=0.55)

fingerid = [4, 8, 12, 16, 20]

while True:
    ret, frame = cap.read()
    if not ret:
        print("Không thể đọc hình ảnh từ camera!")
        break

    frame = detector.findHands(frame)
    lmList = detector.findPosition(frame, handNo=0)

    if len(lmList) != 0:
        fingers = []

        # Kiểm tra ngón cái
        if lmList[fingerid[0]][1] < lmList[fingerid[0] - 1][1]:
            fingers.append(1)
        else:
            fingers.append(0)

        # Kiểm tra 4 ngón còn lại
        for id in range(1, 5):
            if lmList[fingerid[id]][2] < lmList[fingerid[id] - 2][2]:
                fingers.append(1)
            else:
                fingers.append(0)

        songontay = fingers.count(1)
```

```

# Kiểm tra số ngón tay hợp lệ và không bị out of range
if 0 < songontay <= len(lst_2):
    h, w, c = lst_2[songontay - 1].shape
    # Cập nhật vùng ảnh với ảnh ngón tay tương ứng
    frame[0:h, 0:w] = lst_2[songontay - 1]

# Hiển thị số ngón tay nhận diện được
cv2.rectangle(frame, (0, 200), (150, 400), (0, 255, 0), -1)
cv2.putText(frame, str(songontay), (30, 390), cv2.FONT_HERSHEY_PLAIN, 10,
(255, 0, 0), 5)

cTime = time.time()
fps = 1 / (cTime - pTime)
pTime = cTime

# Hiển thị FPS trên màn hình
cv2.putText(frame, f'FPS: {int(fps)}', (150, 70), cv2.FONT_HERSHEY_PLAIN, 3,
(255, 0, 0), 3)

# Hiển thị video
cv2.imshow("NHOM 9", frame)

if cv2.waitKey(1) == ord("q"): # Bấm 'q' để thoát
    break

# Giải phóng camera và đóng cửa sổ
cap.release()
cv2.destroyAllWindows()

+HAND
import cv2
import mediapipe as mp
import time

class handDetector():
    def __init__(self, mode=False, maxHands=2, detectionCon=0.5, trackCon=0.5):
        self.mode = mode
        self.maxHands = maxHands
        self.detectionCon = detectionCon
        self.trackCon = trackCon

        self.mpHands = mp.solutions.hands
        # Thay đổi cách khởi tạo đối tượng Hands() để truyền đúng kiểu dữ liệu
        self.hands = self.mpHands.Hands(
            static_image_mode=self.mode,
            max_num_hands=self.maxHands,
            min_detection_confidence=self.detectionCon,

```

```

        min_tracking_confidence=self.trackCon
    )
    self.mpDraw = mp.solutions.drawing_utils

def findHands(self, img, draw=True):
    imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    self.results = self.hands.process(imgRGB)

    if self.results.multi_hand_landmarks:
        for handLms in self.results.multi_hand_landmarks:
            if draw:
                self.mpDraw.draw_landmarks(img, handLms,
                                             self.mpHands.HAND_CONNECTIONS)

    return img

def findPosition(self, img, handNo=0, draw=True):
    lmList = []
    if self.results.multi_hand_landmarks:
        myHand = self.results.multi_hand_landmarks[handNo]
        for id, lm in enumerate(myHand.landmark):
            h, w, c = img.shape
            cx, cy = int(lm.x * w), int(lm.y * h)
            lmList.append([id, cx, cy])
            if draw:
                cv2.circle(img, (cx, cy), 15, (255, 0, 255), cv2.FILLED)
    return lmList

def main():
    pTime = 0.5
    cap = cv2.VideoCapture(0) # Dùng camera mặc định (0)

    if not cap.isOpened():
        print("Không thể mở camera!")
        return

    detector = handDetector()
    while True:
        success, img = cap.read()
        if not success:
            print("Không thể đọc hình ảnh từ camera!")
            break

        img = detector.findHands(img)
        lmList = detector.findPosition(img)

        if len(lmList) > 4: # Kiểm tra nếu có ít nhất 5 điểm (ngón tay)
            print(lmList[4]) # In vị trí ngón cái (id=4)

```

```
# Tính FPS
cTime = time.time()
fps = 1 / (cTime - pTime)
pTime = cTime

# Hiển thị FPS
cv2.putText(img, f'FPS: {int(fps)}', (10, 70), cv2.FONT_HERSHEY_PLAIN, 3, (255,
0, 255), 3)

cv2.imshow("Hand Tracking", img)

if cv2.waitKey(1) & 0xFF == ord('q'): # Bấm 'q' để thoát
    break

cap.release() # Giải phóng camera
cv2.destroyAllWindows() # Đóng tất cả cửa sổ

if __name__ == "__main__":
    main()
```


Chương 4. KẾT QUẢ VÀ KẾT LUẬN

4.1 Kết quả đạt được

Sau khi thực hiện đề tài, nhóm đã đạt được các mục tiêu đã đề ra:

- Nhận diện các ngón tay: Hệ thống có thể nhận diện được số lượng ngón tay đang giơ lên (từ 0 đến 5 ngón tay). Đây là một yêu cầu cơ bản, phù hợp với các ứng dụng như điều khiển thiết bị thông minh, hoặc nhận diện cử chỉ trong các ứng dụng VR/AR. Nhận dạng chính xác số lần tập động tác thể dục với đúng tư thế bao gồm trực diện và bên hông.
- Mediapipe: Sử dụng thư viện Mediapipe của Google cho việc nhận diện cử chỉ tay. Mediapipe cung cấp các mô hình đã được huấn luyện sẵn để nhận diện các điểm đặc trưng trên tay, từ đó tính toán và phát hiện các cử chỉ tay.
- Giao diện được thiết kế để tạo điều kiện thuận lợi cho người dùng vận hành và kiểm soát.



Hình 4.1: Hiện thị kết quả



Hình 4.2: Hiển thị kết quả



Hình 4.3: Hiện thị kết quả

4.2 Kết luận

Nhìn chung, trong suốt quá trình học tập và nghiên cứu, nhóm đã nỗ lực hoàn thành tốt nhiệm vụ được giao.

Đề tài được triển khai bám sát theo mục tiêu ban đầu và vận dụng hiệu quả những kiến thức đã học trong môn Xử lý ảnh. Mặc dù đề tài vẫn còn thiếu sót rất nhiều, nhưng đối với nhóm vẫn đang còn thiếu kinh nghiệm thực tiễn và kiến thức chuyên sâu nên đây thực sự là một thử thách quan trọng và có ý nghĩa lớn.

Trong quá trình thực hiện đề tài đã giúp nhóm tích lũy thêm nhiều kiến thức, cải thiện hơn về tư duy lập trình cũng như khả năng nghiên cứu. Bên cạnh đó, một số phần trong chương trình vẫn chưa được tối ưu.

Nhóm hy vọng trong tương lai có thể tiếp tục cải tiến hệ thống, mở rộng khả năng nhận diện nhiều loại động tác khác nhau, và ứng dụng vào các bài toán thực tế hiệu quả hơn.

4.3 Hướng phát triển

Vì hệ thống hiện tại đã có thể xác định và đếm số bàn tay theo cách cơ bản, nên nó có thể được phát triển thêm để đáp ứng nhu cầu của mọi người, chẳng hạn như:

1. Cải thiện độ chính xác của mô hình nhận diện

-Sử dụng mạng nơ-ron sâu (Deep Learning): Áp dụng các mô hình học sâu như CNN (Convolutional Neural Networks) hoặc các mô hình học chuyển giao (Transfer Learning) như MobileNet, ResNet để cải thiện độ chính xác trong việc nhận diện các cử chỉ tay phức tạp hơn.

-Sử dụng dữ liệu huấn luyện đa dạng: Để cải thiện khả năng nhận diện trong các điều kiện ánh sáng khác nhau, đa dạng hóa dữ liệu huấn luyện là điều quan trọng. Các video với các cử chỉ tay từ nhiều người, môi trường khác nhau (trong nhà, ngoài trời) sẽ giúp mô hình học được nhiều đặc điểm và cải thiện tính linh hoạt.

2. Ứng dụng thực tế

-Điều khiển thiết bị thông minh (Smart Devices): Đưa ứng dụng nhận diện cử chỉ tay vào việc điều khiển các thiết bị thông minh như TV, điều hòa, máy tính bằng cử chỉ tay thay vì dùng điều khiển từ xa.

-Ứng dụng trong thực tế ảo (Virtual Reality - VR) và thực tế tăng cường (Augmented Reality - AR): Nhận diện cử chỉ tay có thể giúp người dùng tương tác với các môi trường VR hoặc AR mà không cần đến các thiết bị điều khiển vật lý.

-Điều khiển robot: Nhận diện cử chỉ tay có thể dùng để điều khiển robot trong các tình huống đặc biệt, chẳng hạn như trong các ứng dụng cứu hộ, y tế, hoặc thậm chí trong các trò chơi tương tác.

Kết luận:

Nhận diện cử chỉ tay là một lĩnh vực đầy tiềm năng và có thể phát triển theo nhiều hướng khác nhau. Từ việc tối ưu hóa và cải thiện độ chính xác của các mô hình nhận diện đến việc áp dụng vào thực tế để tạo ra các giải pháp thông minh, không chỉ cho các ứng dụng giải trí mà còn trong các lĩnh vực như y tế, giáo dục, công nghiệp, và cuộc sống hàng ngày.

TÀI LIỆU THAM KHẢO

- [1]. Nguyễn Tấn Đồi, Tạ Văn Phương (2022), Giáo trình Thực tập trang bị Điện - Khí nén, Nhà xuất bản Đại học Quốc gia Tp. Hồ Chí Minh.
- [2]. Nguyễn Tấn Đồi (2007), Điều khiển lập trình 1, Khoa điện điện tử, Trường Đại học Sư phạm Kỹ thuật Tp. Hồ Chí Minh.
- [3]. Nguyễn Ngọc Phương, Nguyễn Trường Thịnh (2005), Hệ thống điều khiển tự động khí nén, Khoa Cơ khí chế tạo máy, Trường Đại học Sư phạm Kỹ thuật Tp. Hồ Chí Minh.
- [4]. <https://cellphones.com.vn/sforum/cong-thuc-noi-suy>
- [4]. <https://www.geeksforgeeks.org/python-tkinter-label/>